

# Git & GitHub

## Complete Beginner's Guide

*From zero to confident — install, commit, push, branch, and collaborate*

### What you will learn:

- What Git is and why every developer uses it
- What GitHub is and how it relates to Git
- Installing Git and setting it up for the first time
- The core workflow: init → add → commit → push
- Key concepts: repository, staging area, commit, branch, merge
- Cloning an existing repository from GitHub
- Creating and switching branches
- Merging branches and resolving conflicts
- Pull Requests — the standard way to collaborate
- The most important Git commands with real examples
- A cheat sheet you can keep beside you

## 1 — What Is Git?

Imagine you are writing your thesis. You save a file called *thesis.docx*. The next day you make big changes and save again — the old version is gone. If you realise the new version is worse, you cannot go back. Most people solve this by creating files like *thesis\_v2.docx*, *thesis\_final.docx*, *thesis\_FINAL\_really.docx* — a mess.

**Git solves this problem.** Git is a **version control system** — a tool that tracks every change you make to your files over time. Every time you tell Git to save a snapshot (called a **commit**), Git records exactly what changed, who changed it, and when. You can go back to any snapshot at any time.

★ **Git is a program that runs on your computer.** It lives in a hidden folder called `.git` inside your project folder. It works completely offline — you do not need the internet to use Git.

### What Git tracks:

| Git tracks                             | Git does NOT track                                           |
|----------------------------------------|--------------------------------------------------------------|
| Which lines were added                 | Large binary files (videos, images) — possible but not ideal |
| Which lines were deleted               | Files you tell it to ignore (via <code>.gitignore</code> )   |
| Who made each change                   | Your passwords/API keys (never commit these!)                |
| When the change was made               | Temporary files (build outputs, <code>.o</code> files)       |
| Why you made the change (your message) |                                                              |

## 2 — What Is GitHub?

Git is local — it lives on your computer. **GitHub is a website ([github.com](https://github.com)) that stores a copy of your Git repository online.** Think of it like Google Drive for code, but with Git superpowers.

| Git                                  | GitHub                                                   |
|--------------------------------------|----------------------------------------------------------|
| A program installed on your computer | A website at <a href="https://github.com">github.com</a> |
| Works completely offline             | Requires internet                                        |
| Tracks changes locally               | Stores your history in the cloud                         |
| Free, open source software           | Free for public repos; paid plans for private teams      |
| Command-line tool                    | Has a visual web interface                               |
| Created by Linus Torvalds (2005)     | Founded in 2008, owned by Microsoft since 2018           |

■ Git is the tool. GitHub is the hosting service. Other hosting services exist (GitLab, Bitbucket) but GitHub is the most popular. Everything you learn about Git works with all of them.

## 3 — Key Concepts You Must Know

Before touching any commands, understand these seven terms. Every Git operation is built around them.

### Repository (repo)

A folder that Git is tracking. Contains your files plus the hidden `.git` folder where Git stores all its history. You have a local repo on your computer and a remote repo on GitHub.

## Commit

A saved snapshot of your project at a specific moment. Like taking a photo — you can always go back to any photo. Each commit has a unique ID (a long hex string like a3f9c2b) and a message you write explaining the change.

## Staging area (index)

A preparation zone between your working files and a commit. You 'stage' files to tell Git: include these changes in the next commit. This lets you commit only some of your changes, not everything at once.

## Branch

An independent line of development. The default branch is called 'main' (or 'master' in older repos). You create a new branch to work on a feature without disturbing the main code. Think of it as a parallel universe copy of your project.

## Merge

Combining one branch back into another. When your feature is ready, you merge it into main. Git automatically combines the changes. If the same line was changed on both branches, a conflict occurs.

## Push

Sending your local commits to GitHub (the remote). Until you push, your commits exist only on your computer.

## Pull

Downloading commits from GitHub to your local repo. If a teammate pushed new commits, you pull to get them.

## Clone

Downloading a complete copy of a remote repo (including ALL its history) to your computer for the first time.

★ **Mental model:** Your files on disk → staging area (git add) → local repo (git commit) → GitHub (git push). The opposite direction: GitHub → local repo → your files (git pull).

## 4 — Installing Git and First-Time Setup

### Installing Git:

| Operating system    | How to install                                                                                                                  |
|---------------------|---------------------------------------------------------------------------------------------------------------------------------|
| Windows             | Download from <a href="https://git-scm.com/download/win">git-scm.com/download/win</a> — run the installer, keep default options |
| macOS               | Run: <code>xcode-select --install</code> (installs Xcode command-line tools including Git)                                      |
| Ubuntu/Debian Linux | Run: <code>sudo apt install git</code>                                                                                          |
| Fedora/RHEL Linux   | Run: <code>sudo dnf install git</code>                                                                                          |

### Verify installation:

```
git --version
# Expected output: git version 2.43.0 (or similar)
```

### One-time configuration (do this once after installing):

Git needs to know your name and email — it attaches them to every commit you make.

```
git config --global user.name "Mohamed"
git config --global user.email
"matef517@gmail.com"
git config --global core.editor "nano"
git config --global init.defaultBranch main
```

**--global** — applies to ALL repos on this computer (not just one project). **user.name** — your name as it will appear in every commit. **user.email** — must match your GitHub account email. **core.editor** — which text editor opens for commit messages (nano is easiest for beginners; alternatives: vim, code). **init.defaultBranch main** — makes new repos start with a branch called 'main' instead of 'master'.

### Verify your config:

```
git config --list
# Shows all your settings
```

## 5 — Creating Your First Repository

There are two ways to start: create a new repo from scratch on your computer, or clone an existing repo from GitHub. This section covers creating from scratch.

1

### Create a project folder

`mkdir my_project cd my_project` # You are now inside the project folder

2

### Initialise Git

`git init` # Output: Initialized empty Git repository in .../my\_project/.git/ # Git creates a hidden .git folder — your repo is born

3

### Create a file

`echo "Hello, Git!" > README.md` # Creates a file called README.md with one line of text

4

### Stage the file

`git add README.md` # Tells Git: include README.md in the next commit # Alternative: `git add .` ← stages ALL changed files

5

### Commit the snapshot

`git commit -m "Initial commit: add README"` # Creates your first snapshot # -m means: the message follows in quotes

6

### Check the log

`git log` # Shows your commit history # Each commit has: ID, author, date, message

## 6 — The Staging Area — Why It Exists

Beginners often ask: why is there a staging step? Why not just commit directly? The staging area gives you fine-grained control over what goes into each commit.

Scenario: you changed 5 files. You want commit 1 to contain changes to files A and B, and commit 2 to contain changes to files C, D, E. Without staging, you would have to commit everything at once.

```
# Stage only files A and B:
git add fileA.c fileB.c
git commit -m "Fix: correct tensor shape calculation"

# Now stage the other three:
git add fileC.c fileD.c fileE.c
git commit -m "Feature: add RigL sparse training"
```

Two clean, focused commits instead of one big messy one. Good commits have ONE clear purpose. Future-you (and teammates) will thank you when reading the history.

### git status — your most-used command:

Run this constantly. It tells you the state of everything.

```
git status

# Example output:

On branch main

Changes to be committed: ← staged (will go into next commit)
modified:   fileA.c

Changes not staged for commit: ← modified but NOT staged yet
modified:   fileB.c

Untracked files: ← Git has never seen these
fileC.c
```

★ **Three zones:** Working directory (your actual files) → Staging area (git add) → Repository (git commit). git status shows you where everything is.

## 7 — Connecting to GitHub and Pushing

You have a local repo with commits. Now you want to put it on GitHub so it is backed up online and others can see it.

1

### Create a repo on GitHub

Go to [github.com](https://github.com) → click the '+' button (top right) → New repository. Give it a name. Leave 'Initialize this repository' UNCHECKED (you already have commits locally). Click 'Create repository'. GitHub shows you a page with setup instructions.

2

### Link your local repo to GitHub

`git remote add origin https://github.com/YourUsername/my_project.git` `remote add` — adds a connection to a remote server origin — the conventional nickname for your main remote (you can name it anything) `https://...` — the URL GitHub showed you

**3**

### Push your commits to GitHub

`git push -u origin main -u` — sets 'origin main' as the default; after this, just type 'git push' origin — the remote nickname main — the branch to push Git will ask for your GitHub username and password the first time. Note: GitHub no longer accepts passwords — use a Personal Access Token instead (see Section 8).

**4**

### Verify on GitHub

Refresh `github.com/YourUsername/my_project` — your files should appear!

## 8 — Authentication: Logging into GitHub from the Terminal

Since 2021, GitHub no longer accepts your account password when pushing from the terminal. You must use one of these two methods:

### Method A — Personal Access Token (PAT) — easiest:

1. Go to `github.com` → Settings → Developer settings → Personal access tokens → Tokens (classic).
2. Click 'Generate new token'. Give it a name, set expiry, check the 'repo' scope.
3. Copy the token (you only see it once!).
4. When Git asks for your password, paste the token instead of your password.

### To avoid typing it every time:

```
git config --global credential.helper store
# After you type the token once, Git saves it to disk
```

### Method B — SSH key — more secure, recommended for regular use:

```
# Step 1: Generate an SSH key pair
ssh-keygen -t ed25519 -C "matef517@gmail.com"
# Press Enter 3 times to accept defaults

# Step 2: Copy your PUBLIC key
cat ~/.ssh/id_ed25519.pub
# Copy the output (starts with ssh-ed25519 ...)

# Step 3: Add it to GitHub
# github.com → Settings → SSH and GPG keys → New SSH key → Paste

# Step 4: Test the connection
ssh -T git@github.com
# Output: Hi YourUsername! You've successfully authenticated.

# Step 5: Use SSH URL instead of HTTPS
git remote set-url origin git@github.com:YourUsername/my_project.git
```

## 9 — Cloning a Repository from GitHub

Cloning means downloading a complete copy of an existing GitHub repo to your computer — including every commit, branch, and file in its history.

```
git clone
https://github.com/Owner/RepoName.git
# This creates a folder called RepoName/
# with all files and the full .git history
# inside
cd RepoName
```

**git clone** does three things automatically: 1. Creates a folder named after the repo. 2. Initialises a .git repo inside it. 3. Downloads all commits and sets up 'origin' pointing to GitHub. After cloning you are ready to work immediately — no git init or git remote add needed.

### Clone to a specific folder name:

```
git clone https://github.com/Owner/RepoName.git my_folder
# Downloads into 'my_folder' instead of 'RepoName'
```

→ For your thesis project the clone command would be something like: `git clone https://github.com/YourUsername/OnDeviceTraining.git` This gives you the complete project with all history on any new machine.

## 10 — Branches — Working in Parallel

A branch is an independent copy of your project where you can make changes without affecting the main code. When you are happy with the changes, you merge the branch back.

■ **Rule of thumb:** Never work directly on 'main'. Always create a branch for each feature or fix. main should always contain working, stable code.

### Branch commands:

```
git branch
# Lists all local branches (* = current)
```

Shows which branches exist and which one you are on.

```
git branch feature/rigl-training
# Creates a new branch (does NOT switch to it)
```

Creates a branch named 'feature/rigl-training'. The slash is just a naming convention — it groups related branches visually.

```
git switch feature/rigl-training
# Switches to that branch
# Shortcut: create AND switch in one command:
git switch -c feature/rigl-training
```

**git switch** is the modern command (Git 2.23+). You may also see the older: `git checkout feature/rigl-training` Both do the same thing. Use `git switch` — it is clearer.

```
# After making changes and committing on the
branch:
git push -u origin feature/rigl-training
# Pushes the branch to GitHub
```

Pushes your branch to GitHub so it is backed up and others can see it.

```
git branch -d feature/rigl-training
# Deletes the branch locally (after merging)
```

Clean up merged branches. Use `-D` (capital) to force-delete an unmerged branch.

## 11 — Merging Branches

Merging combines the commits from one branch into another. The typical flow is: work on a feature branch → merge it into main when done.

```
# Step 1: Go to the branch you want to merge INTO (usually main)
git switch main

# Step 2: Merge the feature branch
git merge feature/rigl-training

# Git combines all commits from feature/rigl-training into main

# Step 3: Push the updated main to GitHub
git push
```

### Merge conflicts:

A conflict happens when the same line was changed differently on both branches. Git cannot decide which version to keep — it asks you.

```
# Git tells you:
CONFLICT (content): Merge conflict in main.c
Automatic merge failed; fix conflicts and then commit.

# Open main.c — you will see conflict markers:
<<<<<<< HEAD ← your current branch version
int batchSize = 32;
=====
int batchSize = 64; ← the incoming branch version
>>>>>>> feature/rigl-training

# Edit the file: delete the markers, keep what you want:
int batchSize = 64;

# Then:
git add main.c
git commit -m "Merge: resolve batchSize conflict"
```

## 12 — Pull Requests — Collaborating on GitHub

A Pull Request (PR) is a GitHub feature — it is not a Git command. A PR says: 'I finished work on my branch. Please review it and merge it into main.' PRs are the standard way teams collaborate: one person writes code, others review it.

1

### Push your feature branch to GitHub

```
git push -u origin feature/fqt-quantization
```

2

### Open a Pull Request on GitHub

Go to your repo on github.com. GitHub usually shows a yellow banner: 'Compare & pull request' — click it. Or go to the Pull Requests tab → New pull request. Choose: base = main, compare = feature/fqt-quantization.



3

**Write a description**

Give the PR a title and description explaining: • What you changed • Why you changed it • How to test it Click 'Create pull request'.

4

**Review and discussion**

Teammates can comment on specific lines of code. You can push more commits to the branch — the PR updates automatically.

5

**Merge the PR**

When everyone approves, click 'Merge pull request' on GitHub. GitHub merges the branch into main. You can then delete the feature branch on GitHub (there is a button for it).

6

**Update your local main**

git switch main git pull # Downloads the merge commit — your local main is now up to date

## 13 — Staying Up to Date: pull and fetch

When a teammate pushes commits to GitHub, your local repo does not automatically update. You must manually download the new commits.

```
git pull
# Downloads new commits AND merges them into
your current branch
```

**Most common.** Does two things: download (fetch) + merge. If there are conflicts between their changes and yours, Git asks you to resolve them.

```
git fetch
# Downloads new commits but does NOT merge
them
# You can review them first:
git log origin/main
# Then merge manually:
git merge origin/main
```

**Safer alternative.** Download without touching your files. You can inspect what changed before merging. Good when you want control.

✓ **Daily habit:** Always run `git pull` at the start of your work session before making any changes. This avoids conflicts caused by working on outdated code.

## 14 — .gitignore — Telling Git What to Ignore

Some files should never go into Git: compiled binaries, build outputs, IDE settings, temporary files. The `.gitignore` file lists patterns of files Git should pretend do not exist.

```
# .gitignore for your thesis project

# Build outputs
build/
*.o
*.elf
```

```

*.hex

# Python cache
__pycache__/*
*.pyc

# IDE files
.vscode/
.idea/

# OS files
.DS_Store # macOS
Thumbs.db # Windows

# Secrets – NEVER commit these
*.env
secrets.txt

```

✓ Tip: [github.com/github/gitignore](https://github.com/github/gitignore) has ready-made .gitignore templates for C, Python, CMake, STM32, and many other project types.

## 15 — Undoing Things — Going Back in Time

Git's superpower is that nothing is truly lost. Here are the most common 'undo' operations:

| What you want                         | Command                                          | Notes                                                                       |
|---------------------------------------|--------------------------------------------------|-----------------------------------------------------------------------------|
| Unstage a file (keep changes)         | <code>git restore --staged filename.c</code>     | Removes from staging area but file on disk is unchanged                     |
| Discard changes to a file             | <code>git restore filename.c</code>              | Resets file to last committed version — PERMANENT                           |
| Amend the last commit message         | <code>git commit --amend -m "New message"</code> | Rewrites the last commit (only before pushing)                              |
| Undo last commit, keep changes staged | <code>git reset --soft HEAD~1</code>             | Moves HEAD back 1 commit; changes stay staged                               |
| Undo last commit, unstage changes     | <code>git reset HEAD~1</code>                    | Moves HEAD back; changes go back to working directory                       |
| Undo last commit, DISCARD changes     | <code>git reset --hard HEAD~1</code>             | DANGEROUS: all uncommitted changes lost permanently                         |
| Undo a commit that was pushed         | <code>git revert abc123</code>                   | Creates a NEW commit that undoes abc123 — safe for shared history           |
| See what changed                      | <code>git diff</code>                            | Shows unstaged changes. <code>git diff --staged</code> shows staged changes |

■ **Rule:** Use `git revert` (not `reset`) for commits that have already been pushed to GitHub. `reset` rewrites history and will cause problems for anyone who has already pulled your commits.

## 16 — Complete Command Reference

| Command | What it does |
|---------|--------------|
|---------|--------------|

|                                      |                                                  |
|--------------------------------------|--------------------------------------------------|
| <code>git init</code>                | Create a new local repo in the current folder    |
| <code>git clone</code>               | Download a repo from GitHub                      |
| <code>git status</code>              | Show state of working directory and staging area |
| <code>git add</code>                 | Stage a specific file                            |
| <code>git add .</code>               | Stage ALL changed files                          |
| <code>git commit -m "msg"</code>     | Save staged changes as a commit with message     |
| <code>git log</code>                 | Show commit history                              |
| <code>git log --oneline</code>       | Compact commit history (one line each)           |
| <code>git diff</code>                | Show unstaged changes                            |
| <code>git diff --staged</code>       | Show staged (but not yet committed) changes      |
| <code>git remote add origin</code>   | Connect local repo to GitHub                     |
| <code>git push</code>                | Upload commits to GitHub                         |
| <code>git push -u origin main</code> | Push and set default remote/branch               |
| <code>git pull</code>                | Download + merge latest commits from GitHub      |
| <code>git fetch</code>               | Download without merging                         |
| <code>git branch</code>              | List local branches                              |
| <code>git switch</code>              | Switch to a branch                               |
| <code>git switch -c</code>           | Create and switch to a new branch                |
| <code>git merge</code>               | Merge a branch into the current branch           |
| <code>git branch -d</code>           | Delete a merged branch                           |
| <code>git stash</code>               | Temporarily shelve uncommitted changes           |
| <code>git stash pop</code>           | Restore shelved changes                          |
| <code>git restore --staged</code>    | Unstage a file                                   |
| <code>git restore</code>             | Discard changes to a file (permanent!)           |
| <code>git reset --soft HEAD~1</code> | Undo last commit, keep changes staged            |
| <code>git revert</code>              | Create undo-commit for a pushed commit           |
| <code>git tag v1.0</code>            | Mark the current commit with a version tag       |
| <code>git config --list</code>       | Show all Git configuration settings              |

## 17 — Cheat Sheet — Your Daily Workflow

### Starting work each day:

```
git pull # get latest changes
git switch -c feature/today # create a fresh branch
# ... do your work ...
git status # see what changed
git add . # stage everything
git commit -m "What I did" # save snapshot
```

```
git push -u origin feature/today # send to GitHub
```

### Repeat commit/push cycle as you work:

```
# Commit often – small, focused commits are better than one huge one
git add specific_file.c
git commit -m "Fix: tensor shape off-by-one in calcNumberOfElements"
git push
```

### Key terms at a glance:

|                          |                                                                   |
|--------------------------|-------------------------------------------------------------------|
| <b>Repository (repo)</b> | Your project folder tracked by Git                                |
| <b>Commit</b>            | A saved snapshot with a message and ID                            |
| <b>Staging area</b>      | The preparation zone before committing (git add)                  |
| <b>Branch</b>            | An independent line of development                                |
| <b>main</b>              | The default main branch — keep it always stable                   |
| <b>origin</b>            | The nickname for your GitHub remote                               |
| <b>HEAD</b>              | Pointer to the commit you are currently on                        |
| <b>Push</b>              | Send local commits → GitHub                                       |
| <b>Pull</b>              | Download + merge GitHub commits → local                           |
| <b>Clone</b>             | Download a complete copy of a GitHub repo                         |
| <b>Merge</b>             | Combine one branch into another                                   |
| <b>Pull Request</b>      | GitHub feature: propose merging your branch, get review           |
| <b>Conflict</b>          | Same line changed on both branches — you must resolve it manually |
| <b>.gitignore</b>        | File listing patterns Git should never track                      |
| <b>git stash</b>         | Temporarily shelve changes without committing                     |